

AKADEMIA LEONA KOŹMIŃSKIEGO · INFORMATYKA, I ROK · 08.01.2027

Lab 12

Obsługa wyjątków

dr inż. Karol Chlasta · WarsawIQ

Repozytorium kursu: github.com/KarolChlasta/KU-PyDev

Cel zajęć

- Rozumieć **mechanizm wyjątków**: co się dzieje, gdy program napotyka błąd
- `try` / `except` / `else` / `finally`, `raise`, własne klasy wyjątków
- Asercje i różnica między **błędem użytkownika** a **błędem programisty**
- Zabezpieczyć kalkulator, walidację danych i operacje na plikach
- Efekty z sylabusa: **U1**(skrypty z obsługą wyjątków), **KS1**

CZĘŚĆ I

Teoria: wyjątki

Błąd składni a wyjątek

	BŁĄD SKŁADNI	WYJĄTEK
Kiedy	przed uruchomieniem (kompilacja do kodu bajtowego)	w trakcie działania
Przykład	<code>print("Hi"</code>	<code>10 / 0, int("abc")</code>
Da się obsłużyć?	nie – popraw kod	tak – <code>try</code> / <code>except</code>

```
Traceback (most recent call last):
  File "kalk.py", line 3, in <module>
    print(a / b)
    ~~~^~~~
ZeroDivisionError: division by zero
```

- Traceback czytaj **od dołu**: typ i opis błędu, potem miejsce, potem ścieżka wywołań

Hierarchia wyjątków (fragment)

```

BaseException
├── KeyboardInterrupt          ← Ctrl+C
├── Exception
│   ├── ArithmeticError
│   │   ├── ZeroDivisionError
│   │   └── OverflowError
│   ├── LookupError
│   │   ├── IndexError
│   │   └── KeyError
│   ├── OSError
│   │   ├── FileNotFoundError
│   │   └── PermissionError
│   ├── TypeError
│   └── ValueError

```

try / except / else / finally

```
try:
    value = float(text)           # kod, który może zawieść
except ValueError:
    print("To nie jest liczba")   # tylko gdy wystąpił ValueError
else:
    print(value * 2)              # tylko gdy NIE było wyjątku
finally:
    print("koniec")              # ZAWSZE – nawet przy return albo innym wyjątku
```

SYTUACJA	EXCEPT	ELSE	FINALLY
brak błędu	–	✓	✓
ValueError	✓	–	✓
inny wyjątek	–	–	✓, potem wyjątek leci dalej

Zgłaszanie wyjątków: `raise`

```
def parse_age(text):  
    try:  
        age = int(text)  
    except ValueError:  
        raise ValueError("Wiek musi być liczbą całkowitą") from None  
    if age < 0:  
        raise ValueError("Wiek nie może być ujemny")  
    return age
```

- Funkcja **nie wie**, jak zareagować na błąd (okno dialogowe? log? ponowne pytanie?) – zgłasza wyjątek, a decyzję podejmuje **wywołujący**
- `raise` bez argumentu w `except` – przekaż ten sam wyjątek dalej
- `raise ... from err` – zachowaj przyczynę (łańcuch wyjątków); `from None` – ukryj ją

Własne klasy wyjątków

```
class InsufficientFundsError(Exception):
    def __init__(self, balance, amount):
        self.balance = balance
        self.amount = amount
        super().__init__(f"Brak środków: saldo {balance}, żądano {amount}")

try:
    withdraw(100, 150)
except InsufficientFundsError as e:
    print(e, e.amount - e.balance)    # Brak środków: saldo 100, żądano 150  50
```

- Nazwa wyjątku opisuje **problem w dziedzinie** (*brak środków*), nie szczegół techniczny
- Hierarchia własnych wyjątków (`PeselError` → `PeselChecksumError`) pozwala łapać ogólnie albo szczegółowo

Asercje

```
def percent(part, whole):  
    assert whole > 0, "Całość musi być dodatnia"  
    return part / whole * 100
```

- `assert` sprawdza **założenie programisty** („to się nie powinno zdarzyć”)
- Python uruchomiony z flagą `-O` **pomija** asercje → nigdy nie używaj ich do walidacji danych użytkownika

SYTUACJA	NARZĘDZIE
użytkownik wpisał „abc” zamiast liczby	<code>if / try</code> + komunikat, <code>ValueError</code>
funkcja wewnętrzna dostała ujemną długość boku	<code>assert</code>
test sprawdza wynik	<code>self.assertEqual</code> (lab 14)

Dobre praktyki

- **Łap konkretne wyjątki:** `except ValueError`, nie gołe `except:` (złapie nawet Ctrl+C)
- **Nie połykaj błędów:** `except Exception: pass` ukrywa problem, zamiast go rozwiązać
- **Mały blok `try`:** tylko linie, które mogą zawieść
- **EAFP** („łatwiej prosić o wybaczenie niż o pozwolenie”) – styl typowy dla Pythona:

```
# LBYL - „patrz, zanim skoczysz”      # EAFP - „spróbuj i obsłuż błąd”
if key in data:                        try:
    value = data[key]                  value = data[key]
else:                                  except KeyError:
    value = None                       value = None
```

CZĘŚĆ II

Praktyka

Testowalne wczytywanie danych

```
def read_int(prompt, low, high, input_func=input, output=print):  
    while True:  
        try:  
            value = int(input_func(prompt))  
        except ValueError:  
            output("Podaj liczbę całkowitą")  
            continue  
        if low <= value <= high:  
            return value  
        output(f"Wartość spoza zakresu {low}-{high}")
```

- `input_func` i `output` jako parametry → test podstawia **atrapy** zamiast klawiatury i ekranu
- To samo podejście co z `today` w labie 11 – przygotowanie do mocków w labie 14

Ciekawostki 🐛

- Python 3.10 zaczął podpowiadać w błędach „*Did you mean...?*”, a 3.11 zaznacza w tracebacku dokładne miejsce błędu znakami `~~^~~` (PEP 657)
- Python 3.11 dodał **grupy wyjątków** i `except*` (PEP 654) – do obsługi wielu błędów naraz, np. w programach współbieżnych
- **Tony Hoare** nazwał wprowadzenie referencji pustej (*null*) w 1965 r. swoim „*błędem za miliard dolarów*” (QCon, 2009) – w Pythonie jej odpowiednikiem jest `None` i `AttributeError: 'NoneType'...`
- Hasło EAFP przypisuje się **Grace Hopper**

Dzisiejsze zadania

PLIK	ZADANIE
★ <code>zad1_kalkulator.py</code>	bezpieczny kalkulator
★ <code>zad2_walidacja.py</code>	walidacja danych
★ <code>zad3_pliki.py</code>	konfiguracja, <code>finally</code>
★ <code>zad4_wyjatki.py</code>	własne wyjątki i asercje
☆ <code>zad5_pesel.py</code>	walidator PESEL

Sprawdzenie i oddanie

```
cd lab12_wyjatki
python -m unittest discover -s tests -v
git add .
git commit -m "Lab 12"
git push
```

- Dziś wyjątkowo **piątek 14:00–17:15, sala A/124**
- Punkt aktywności: **obecność + wszystkie testy ★ zielone + push na GitHub przed 17:15**

Do poczytania

- E. Matthes, *Python. Instrukcje dla programisty*, Helion 2023 – rozdz. 10
- Dokumentacja: *Errors and Exceptions, Built-in Exceptions* (hierarchia)
- Słownik Pythona: hasła *EAFP* i *LBYL* (docs.python.org/3/glossary.html)

Następny lab (12.01): sortowanie, wyszukiwanie, notacja Big O + blok AI 